

# CloudForensiX

A forensic-ready monitoring and incident-response framework for multi-tenant clouds

**Aditya Sharma** · **Atithi Jaiman** · **Ashish Singh** · **Chiranthan**

Cloud and SRE Security — Monitoring and Incident Response

Base paper: Manral et al., "A Systematic Survey on Cloud Forensics", ACM Computing Surveys, 2019

github.com/4uashish2628/cloudsre

# Why investigating a cloud incident is hard

Aditya



## Unknown evidence location

Data is virtualised and spread across data centres and jurisdictions. No physical seizure.



## Shared, multi-tenant resources

One host, one hypervisor log, one memory dump can hold many customers' data.



## Scattered logs and clocks

User, tenant and provider each keep different log formats, time zones and drifting clocks.



## Trust in the provider

The provider controls the logs. A malicious insider could edit them.

# Base paper: a survey of cloud forensics

Aditya

**Manral, Somani, Choo, Conti, Gaur**

ACM Computing Surveys 52(6), 2019

- 80 papers (2009–2019) classified into a taxonomy
- Table 14: which artifacts each stakeholder holds
- 5 investigation guidelines (Sec. 8.2)

**Incident-driven**
continuous / post-incident

**Provider-driven**
log- and agent-based

**Consumer-driven**
physical / management plane

**Resource-driven**
VM, storage, network



## Gaps the survey points out

Most proposals "lack implementation and model evaluation", and there are no metrics to compare forensic solutions.

# Our idea: guidelines into one working system

Aditya

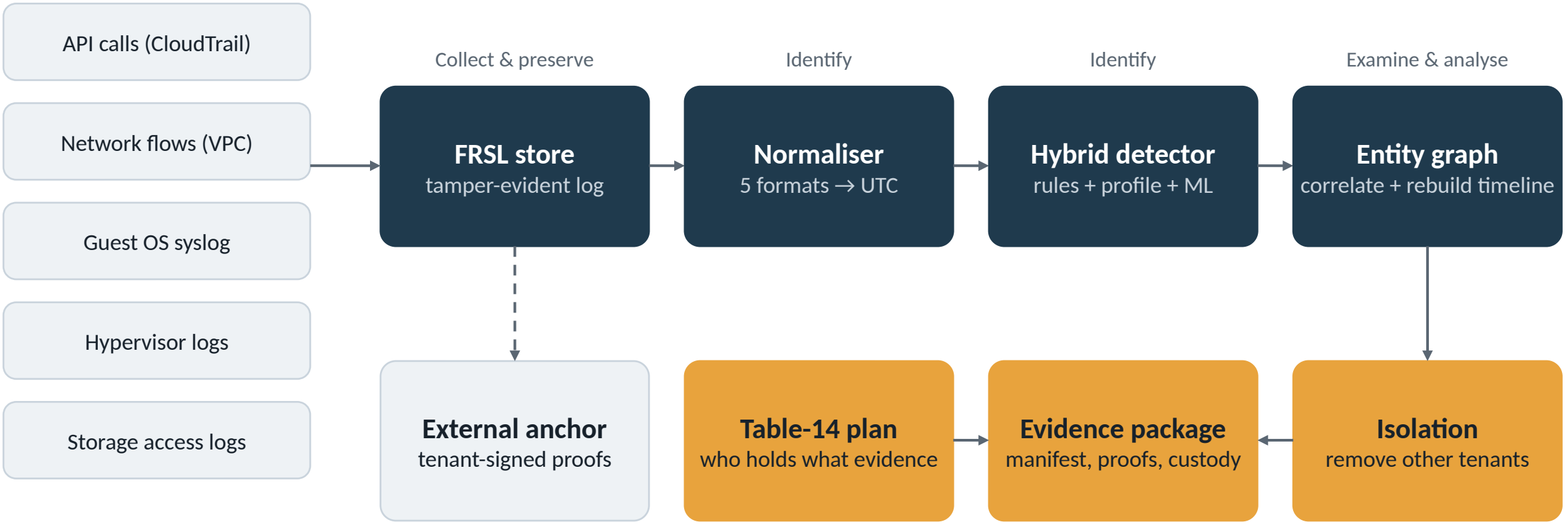
| Requirement (survey section) | SecLaaS | BlockSLaaS | FROST | FMP | CLOSER | Ours |
|---|---|---|---|---|---|---|
| R1 Tamper-evident logs, safe from a malicious provider (3.1, 8.2.3) | ◐ | ◐ | – | ◐ | – | ✓ |
| R2 Aggregate logs, fix time zones (8.2.2) | – | – | – | – | ◐ | ✓ |
| R3 Live detection, few false alerts (8.2.1, 7.2) | – | – | – | ◐ | – | ✓ |
| R4 Correlation and timeline reconstruction (4.1) | – | – | – | – | ✓ | ✓ |
| R5 Privacy and evidence isolation (8.2.4) | – | – | ◐ | – | – | ✓ |
| R6 Integrity, custody, metrics (5, 7.2, 8.2.5) | ◐ | ◐ | ◐ | ◐ | – | ✓ |

✓ addressed ◐ partially – not addressed

**Six requirements → six components, each tested against the methods the survey reviews.**

# Architecture of CloudForensiX

Atithi




Stages follow the survey's five-step investigation process. Orange = what the investigator receives.

# Who we defend against

Atithi



## External attacker

Steals credentials or breaks into VMs; runs one of five attack campaigns.

***Detect fast, rebuild the full story***



## Malicious cloud insider

Full write access to the log store. Can edit, delete, reorder or truncate records and recompute every hash.

***Cannot forge the tenant's key or rewrite published anchors***



## Curious co-tenant

Should never see another customer's data in the evidence.

***Isolate and pseudonymise***

# FRSL: a log the provider cannot quietly rewrite

Atithi

1 

**Hash chain**
Each entry includes the hash of the previous one, so order and deletions are protected.

2 

**Merkle-sealed batches**
Every B entries are sealed under one Merkle root; one record proves itself with 10 hashes.

3 

**Proof of Past Log, signed by the tenant**
Root, count and last hash are MAC'd with a key only the tenant holds.

4 

**Published outside the provider**
Proofs go to an append-only anchor. A Bloom filter answers "did this record exist?"

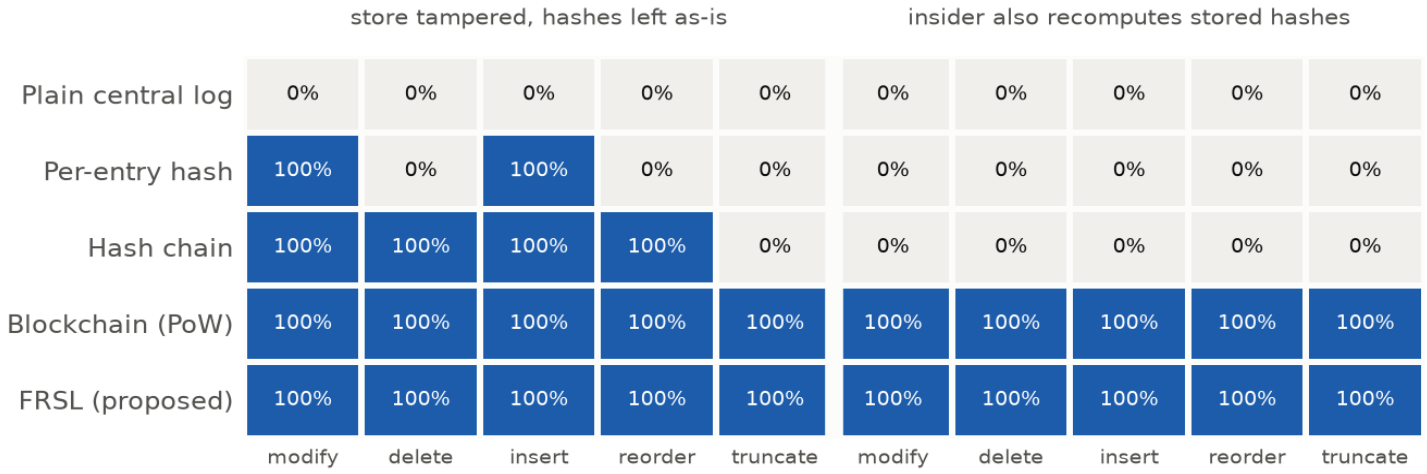


*The provider can rewrite its own disk, but not the tenant's anchor.*

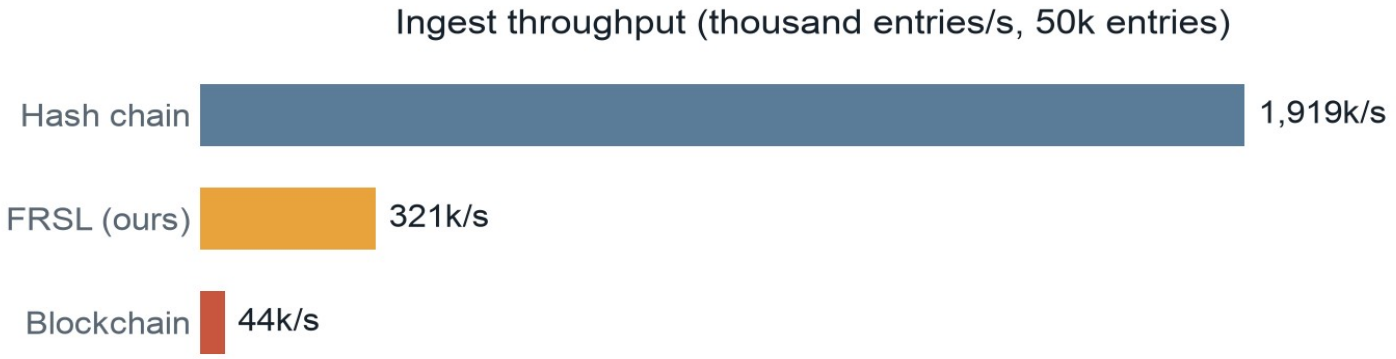
Fixes the weakness the survey lists for SecLaaS and BlockSLaaS: no protection against a malicious provider, and high cost.

# FRSL results: tamper detection and cost

Atithi




Detection rate over 100 trials per attack. Only FRSL and blockchain catch an insider who recomputes hashes.




**100%**

of insider attacks detected, even with recomputed hashes

**7.2×**

faster ingest than blockchain logging

**8 µs**

to verify one record (5,300× faster than a full audit)

*Caveat: records not yet sealed can be cut off undetected → seal on a timer.*

# Test setup: a simulated multi-tenant cloud

Ashish

**20**

attack campaigns per 24-hour run

**5**

random seeds, results averaged

**37k**

raw log records per run (4 tenants)

### 5 attack types

- Password brute force → data theft
- Privilege escalation to admin
- Cryptomining on stolen GPUs
- SSH break-in → lateral movement
- Anti-forensics: logging switched off, logs shredded

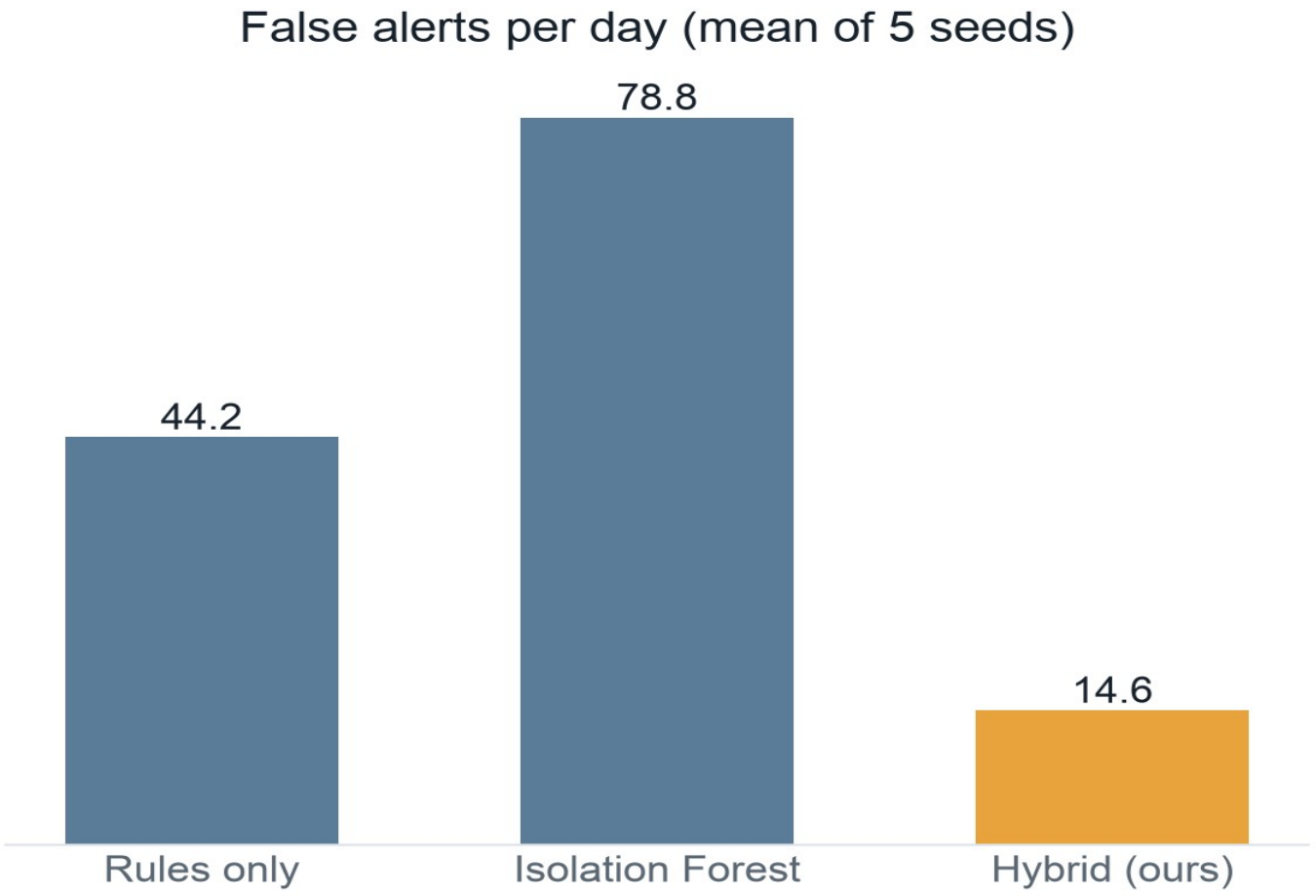
### Harmless look-alikes

- Forgotten passwords, travelling users
- Nightly backups with bulk downloads
- Disaster-recovery drill from a new site
- Real admin grants, GPU batch jobs
- Internet SSH scanners

Why simulated: researchers cannot instrument real providers' back-ends (the survey says so too). Logs use real formats; hidden ground truth is used only for scoring.

# Hybrid detection: 67% fewer false alerts

Ashish




**100%**

of campaigns detected (Isolation Forest alone: 26%)

**83 s**

mean time to detect

**0.49 → 0.78**

event precision, rules vs hybrid

How: rules fire, then alerts matching learnt normal behaviour (known user + known IP) are suppressed, and strong anomalies are added.

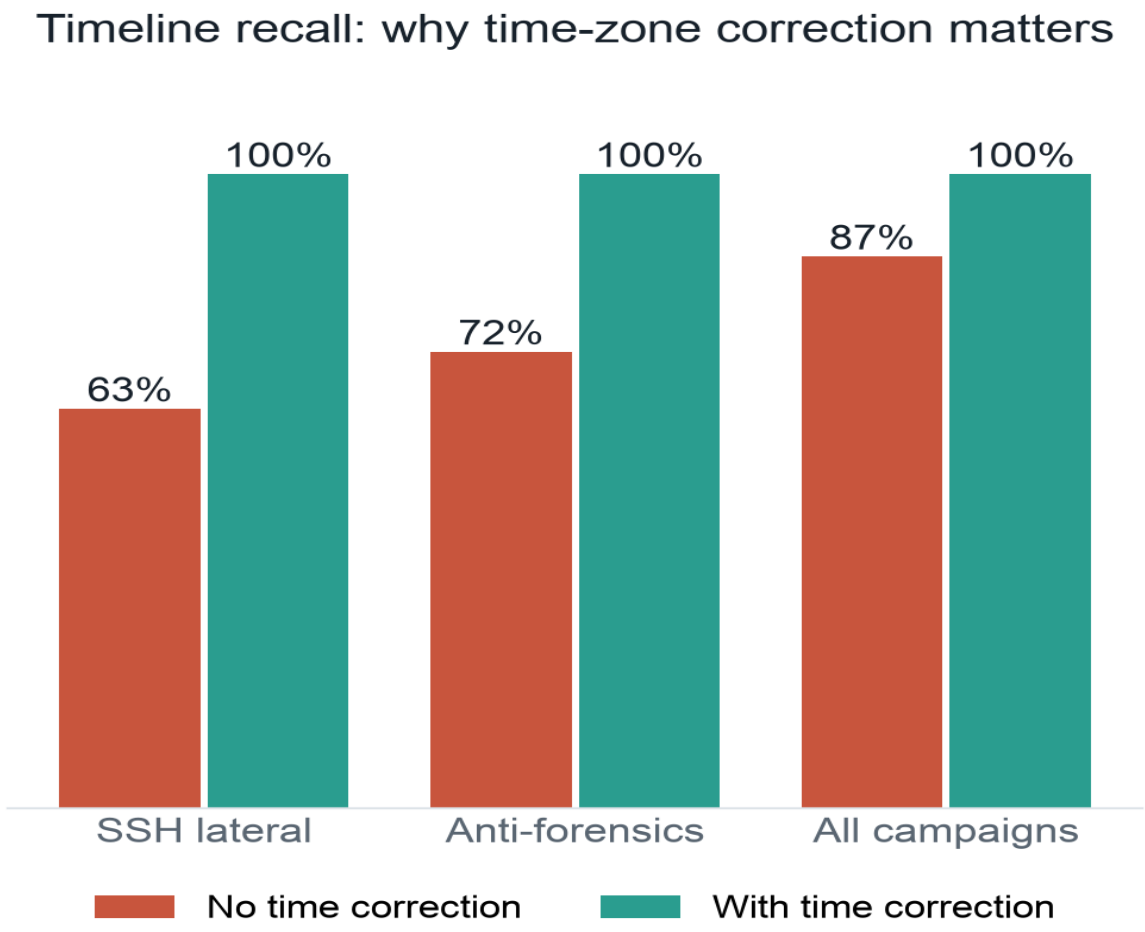
# Rebuilding the full attack timeline

Ashish

**34%** → **100%**

of each attack's events: alerts alone vs. after entity-graph reconstruction

**Entity graph (ours)**
Links alerts sharing an attacker IP, user or host. Best grouping score (ARI 0.9996) vs. Leader-Follower [CLOSER] and a time window; splits campaigns least (1.19 incidents each vs 1.39–1.85).



# Live demo

Ashish

```
$ cd cloudsre
$ .venv/bin/python -m pytest -q
28 passed

$ .venv/bin/python -m cloudforensix.cli \
      investigate --seed 7 --out cases
alerts 116 · incidents 24 · campaigns 20/20
store_verified: true
```

## Open an incident report and point at:

**Timeline**
failed SSH logins → success → dropper → lateral movement

**Integrity**
50/50 records verified by Merkle proof

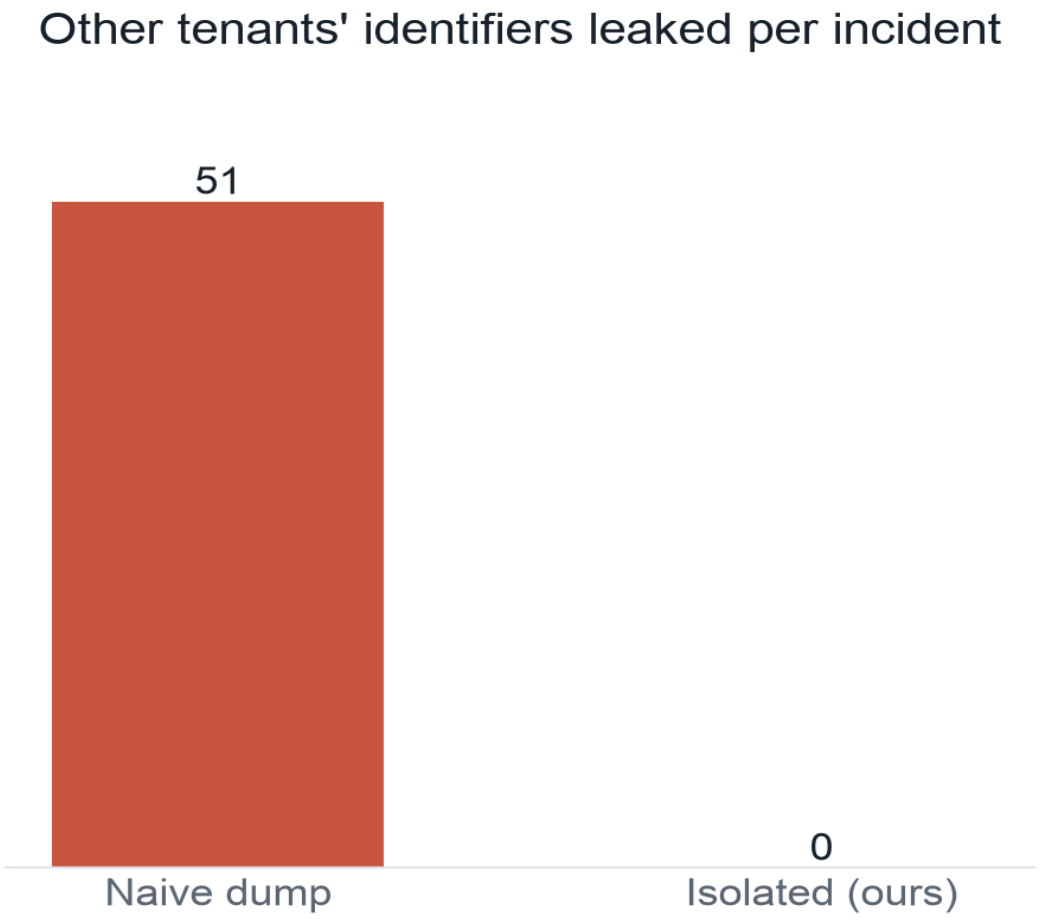
**Privacy**
residual foreign identifiers = 0

**Acquisition plan**
who holds each piece of evidence (Table 14)

Backup: screen recording + examples/CASE-tenant-a-020 in the repo

# Evidence isolation: no other customer's data

Chiranthan




**51 → 0**

foreign-tenant identifiers per incident (119 incidents)

**100%**

of the attack's own records kept

**7,150 → 2,030**

records per evidence set: smaller and focused

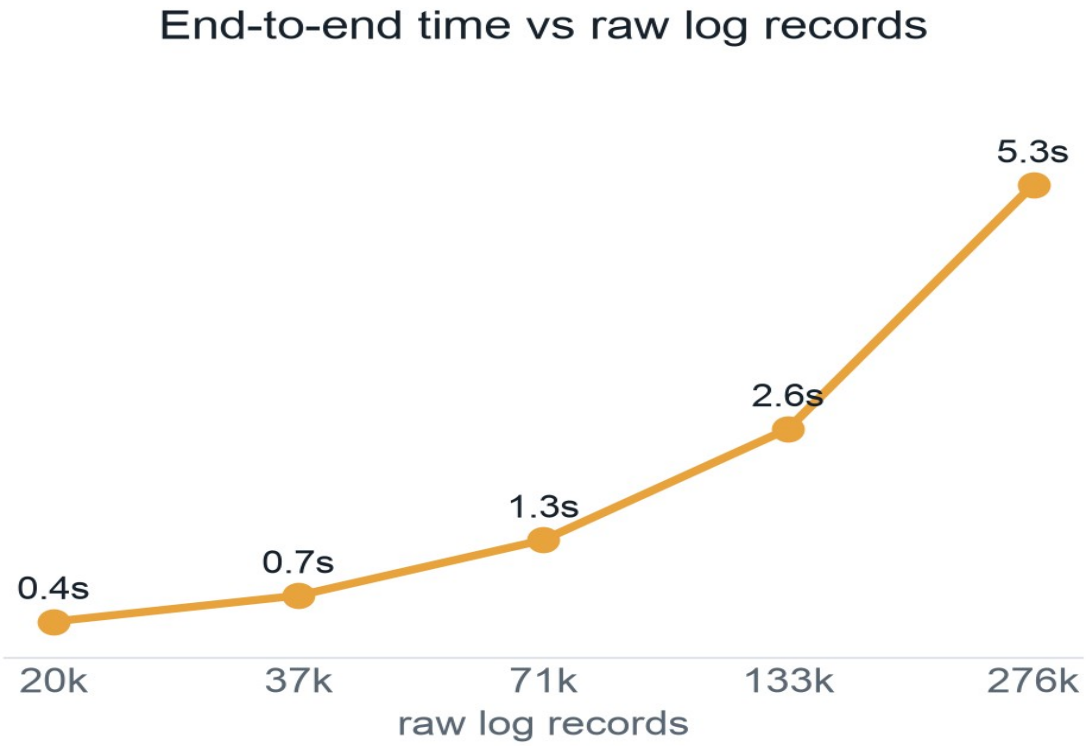
Other tenants' records are dropped; shared host logs keep only our VMs, with the rest replaced by keyed pseudonyms a court can unlock.

# Provider dependency and scalability

Chiranthan

| Attack | Needs provider | Wait today* |
|---|---|---|
| Data theft | storage logs | ≈80 h |
| Cryptomining | hypervisor logs | ≈92 h |
| Privilege escalation | – | ≈5 h |
| SSH lateral | – | ≈8 h |
| Anti-forensics | – | ≈8 h |

Using the survey's Table 14. With our pipeline, owner-side evidence is ready in seconds; only a provider forensics-as-a-service API removes the long wait for provider-held logs.




**≈52k records/s**

linear scaling on one laptop core

** modelled median, assumed delays (owner ≈4 h, legal request to provider ≈72 h)*

# What we contributed, and its limits

Chiranthan

## Contributions

- 6 requirements derived from the survey
- Open-source framework: ~2,300 lines, 28 tests
- FRSL: secure log safe from a malicious provider
- Reproducible simulator with ground truth
- Forensic metric suite + 8 experiments

## Limitations

- Simulated data, not a real provider
- Rules written knowing the attack types
- Tenant MAC, not public signatures
- Unsealed records can be truncated
- Acquisition delays are modelled

# Three takeaways



## External anchoring

Defeats a malicious insider at a fraction of blockchain's cost.



## Context beats raw rules

Learnt behaviour cuts false alerts by two thirds, no campaign missed.



## Correlate with correct time

Turns a third of the evidence into all of it.

Future work: real cloud log datasets · public signatures with TPM keys · streaming on OpenSearch · container and SDN logs

# Thank you — questions?

github.com/4uashish2628/cloudsre